

Node.js Training Material

Contents

1. Node JS Intro
2. How Node.js Works
3. Installation and Code Editor Setup
4. Your First Node.js Script
5. Understanding Modules
 - 5.1 Built-in Modules (Core Modules)
 - 5.2 User-Defined Modules
 - 5.3 Third-Party Modules
6. Async and Await (Understanding Asynchronous Code)
7. Introduction to Express.js
 - 7.1 Key Characteristics: -
 - 7.2 Core Functionality: -
 - 7.3 What's Happening Here (Simple Understanding)
8. Folder Structure in Node.js
 - 8.1. Layer-Based Structure (Basic / Beginner Friendly)
 - 8.2. Feature-Based Structure (Industry Standard)
 - 8.3 Understanding Project Folders (with Examples):
 - 8.4 CommonJS vs ES Modules (Important Concept)
9. Commonly Used Third-Party Dependencies in Node.js
10. CRUD with Register & Login (Basic Project)
 - 10.1 Install Required Dependencies: -
 - 10.2 Project Setup:-
11. Environment Variables (.env) in Node.js

1. Node JS Intro:

Node.js is a runtime environment that allows JavaScript to **run outside the browser**, mainly for building backend applications. Built on **Chrome's V8 engine**, it executes code quickly and efficiently.

With Node.js, developers can use JavaScript to create APIs, handle server logic, and work with databases. Its asynchronous and non-blocking nature helps handle multiple requests at the same time, making it suitable for fast and scalable applications like real-time systems and modern web apps.

👉 Normally:

- JavaScript runs in browser (for buttons, UI, etc.)

👉 With Node.js:

- JavaScript runs on the server
- You can build APIs, handle data, connect databases

2. How Node.js Works:

Node.js works using an event-driven, non-blocking architecture that allows it to handle multiple requests efficiently.

- **Step 1: Client sends a request**
A user (browser or app) sends a request to the Node.js server.
- **Step 2: Request enters the Event Loop**
The request is received and placed in the event loop, which is the core mechanism that manages all operations.
- **Step 3: Check for blocking or non-blocking task**
If the task is simple (like returning data), Node.js processes it immediately.
If it involves time-consuming work (like database or file access), it is treated as a non-blocking task.
- **Step 4: Delegate heavy tasks**
Node.js sends heavy tasks to the system (background workers / APIs) instead of waiting.
- **Step 5: Continue handling other requests**
While the heavy task is processing, Node.js does not wait—it keeps handling new incoming requests.
- **Step 6: Task completion (Callback/Promise)**
Once the background task is done, the result is returned to Node.js using a callback, promise, or async/await.
- **Step 7: Send response to client**
Node.js sends the final response back to the user.

3. Installation and Code Editor Setup:

- To start working with [Node.js](#), you need to install it on your system.
- Download the LTS (Long Term Support) version from the official Node.js website and follow the installation steps.
- This will also install npm (Node Package Manager), which is used to manage libraries and dependencies.

After installation, open the terminal or command prompt and verify it using:

- o `node -v` (Eg: v20.19.4)
- o `npm -v` (Eg: 11.12.1)

If version numbers are displayed, Node.js is successfully installed.

- For writing and managing code, you will need a code editor. A widely used option is [Visual Studio Code](#), which is lightweight and provides useful features like extensions, debugging, and built-in terminal support. You can download and install it, then open your project folder directly inside the editor to start writing Node.js programs.

4. Your First Node.js Script:

Once Node.js is installed, you can run your first JavaScript program directly on your system. **Node.js** scripts are created with the `js` file extension. Remember that Node.js is not a programming language. All the code in this course is JavaScript code, which is why the `js` extension is used.

Below is an example script stored in a file named **index.js**.

```
console.log('Hello Node.js!')
```

Running a Script:

- Now open your terminal, navigate to the folder where the file is saved, and run:
`node index.js`
- You will see the output:
`Hello Node.js!`

When a Node.js script calls `console.log`, the logged values will show up in the terminal. This is a great way to get output from your Node.js application

5. Understanding Modules:

In **Node.js**, a module is simply a way to split your code into smaller, reusable parts. Instead of writing everything in one file, you organize functionality into separate files and use them wherever needed. This makes your code cleaner, easier to manage, and scalable as your application grows.

Every file in Node.js is treated as a module. To use code from one file in another, Node.js provides **require()** to import and **module.exports** to share functionality.

For example, you can create a module that contains a function named as `math.js`:

```
function add(a, b) {  
  return a + b;  
}  
module.exports = { add };
```

Then use it in another file:

```
const math = require("./math"); ---- Importing  
console.log(math.add(2, 3));
```

Here, the **add** function is defined in one file and reused in another. This approach avoids repeating code and helps organize large applications into meaningful parts.

Node.js also provides built-in modules (like file system or OS utilities) and allows you to install third-party modules using npm. Together, these modules form the building blocks of any Node.js application.

Types of Modules in Node.js

In Node.js, a *module* means a file or package that contains reusable code. Modules are mainly divided into three types:

- Built-in Modules (Core Modules)
- User-Defined Modules
- Third-Party Modules

5.1 Built-in Modules (Core Modules)

These are modules already provided by Node.js. You don't need to install them.

Eg:-

```
const os = require("os");
console.log(os.platform());
```

👉 Used for system-level operations like file handling, OS info, etc.

👉 Examples: *fs, http, path, os*

5.2 User-Defined Modules

These are modules created by you to organize and reuse your code.

Eg: -

```
// math.js
function add(a, b) {
  return a + b;
}
module.exports = { add };
```

Using it:

```
const math = require("./math");
console.log(math.add(2, 3));
```

5.3 Third-Party Modules

These are modules installed from npm (Node Package Manager).

Eg:-

```
const express = require("express");
npm install express - installation cmd
```

👉 Used to avoid writing everything from scratch

👉 Examples: *Express, Mongoose, Axios*

Final Understanding (Important)

👉 Module = reusable code (file or package)

👉 Types:

- Built-in (already available)

- User-defined (created by you)
- Third-party (installed using npm)

6. Async and Await (Understanding Asynchronous Code)

In Node.js, many operations like database calls, API requests, and file handling take time to complete. These are called **asynchronous operations**.

Instead of blocking the program and waiting, Node.js continues running other code and handles the result later.

6.1 What is async?

async is used to define a function that works with asynchronous operations.

```
async function getData() {
    return "Hello";
}
```

It always returns a promise.

6.2 What is await?

await is used inside an async function to **wait for a result** before moving to the next line.

```
async function getData() {
    let result = await someAsyncTask();
    console.log(result);
}
```

It makes async code look like normal (synchronous) code.

6.3 Without async/await (Confusing way)

```
User.find().then((data) => {
    console.log(data);
});
```

6.4 With async/await (Clean way)

```
const users = await User.find();
console.log(users);
```

7. Introduction to Express.js :

Express.js is a lightweight framework built on top of Node.js that makes it easier to create servers and APIs. Instead of writing complex low-level code, Express gives you simple methods to handle routes, requests, and responses.

7.1 Key Characteristics: -

- **Minimalist & Unopinionated:** It provides only the essential tools (routing, middleware, and HTTP helpers), allowing you to structure your application however you prefer.
- **Middleware-Based:** The entire framework is essentially a series of middleware function calls that execute in sequence during the request-response cycle.
- **Fast Performance:** Because it is a thin layer over Node.js, it retains Node's high-performance, non-blocking characteristics.
- **Database Integration:** Express facilitates communication between your application and databases like MongoDB or PostgreSQL.
- **API Development:** It is widely used to create RESTful APIs that serve data to frontend applications like React, Angular, or Vue.js.
- **Server-Side Routing:** It manages how the server responds to specific URL requests (e.g., GET /users or POST /login).

7.2 Core Functionality: -

- **Routing:** Defines how your application responds to client requests at specific endpoints (URIs) using HTTP methods like GET, POST, PUT, and DELETE.
- **Middleware Support:** Allows you to plug in code to handle tasks such as parsing JSON payloads, logging, user authentication, and error handling before the request reaches its final destination.
- **Dynamic Rendering:** Integrates with over 14 template engines (like EJS, Pug, or Mustache) to generate dynamic HTML content on the server.
- **Static File Serving:** Includes built-in methods like `express.static()` to serve images, CSS, and JavaScript files directly from a directory.

First, install Express in your project:

- `npm init -y` --- Create Project
- `npm install express`

Now create a file **server.js** and write a basic server:

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.send("Hello from Express Server");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Run the server using this command: - **node server.js**

Output : -

- Open your browser and go to:
- Run this on your browser :- <http://localhost:3000>
- You will see: Hello from Express Server.
- This shows that your server is running and responding to requests.

7.3 What's Happening Here (Simple Understanding)

- `express()` creates a server
- `app.get()` handles a request from the browser
- `res.send()` sends a response
- `app.listen()` starts the server

Clear Explanation: -

When you write an Express program using **Express.js** on top of **Node.js**, you are basically creating a system that listens for requests and sends responses.

First, when you call **`express()`**, you are creating an application object. Think of this as setting up your server—this object will handle all incoming requests and decide how to respond.

Next, **`app.get("/", ...)`** defines a route. This means: *"If someone opens the homepage (/) using a browser, run this function."* The function receives two important things:

- **`req (request)`**: information coming from the user
- **`res (response)`**: what you will send back

Inside that function, **`res.send()`** is used to send a response back to the client. This could be text, HTML, JSON data, or anything else. Once **`res.send()`** is called, the request is completed.

Finally, **`app.listen(3000)`** starts the server and tells it to listen for incoming requests on port 3000. Until this line runs, your server is not active.

Simple Flow:-

Browser → Request → Express Server → Route Match → Send Response → Browser shows result.

8.Folder Structure in Node.js (Best Practice)

There are **two common approaches**:

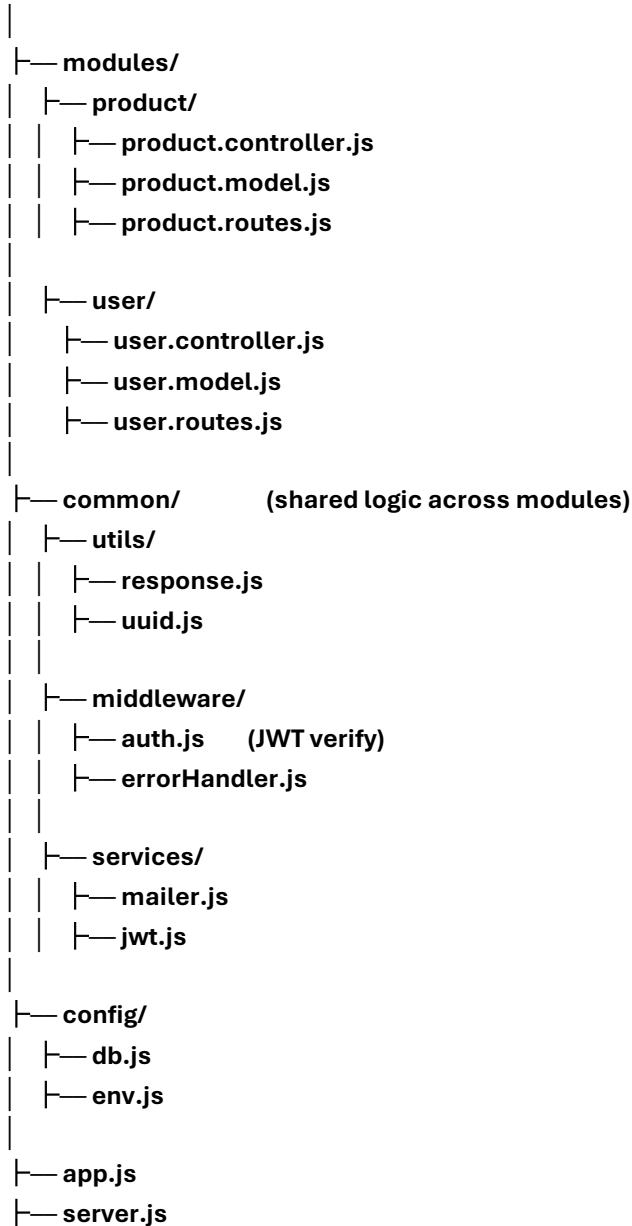
- 1. Layer-Based Structure (Basic / Beginner Friendly)
- 2. Feature-Based Structure (Industry Preferred)

8.1. Layer-Based Structure (Basic / Beginner Friendly)

```
project/
|
|— controllers/
|— models/
|— routes/
|
|— utils/      (helper functions)
|   |— response.js (success & error format)
|   |— uuid.js
|
|— middleware/
|   |— auth.js   (JWT verification)
|   |— errorHandler.js
|
|— services/
|   |— mailer.js (email logic)
|   |— jwt.js   (JWT creation & verify)
|
|— config/
|   |— db.js    (database connection)
|   |— env.js
|
|— app.js
|— server.js
```

8.2. Feature-Based Structure (Industry Standard)

project/



Meaning:

- **controllers/** → business logic (functions)
- **models/** → database schema (MongoDB, etc.)
- **routes/** → API endpoints (GET, POST, etc.)

- **app.js** → express app configuration
- **server.js** → starts the server
- **utils/** → small reusable helpers (format response, generate IDs)
- **middleware/** → runs before controller (auth check, error handling)
- **services/** → core logic like sending email, creating JWT
- **config/** → DB connection, environment variables

8.3 Understanding Project Folders (with Examples):

- **Controllers/**

This folder contains the business logic—what should happen when a request comes in.

Eg:- // controllers/user.controller.js

```
exports.getUsers = (req, res) => {
  res.send("All users data");
};
```

It handles what to do, not how the request comes

- **Models/**

This defines the database structure (schema).

Eg: - // models/user.model.js

```
const user = {
  name: String,
  email: String
};
module.exports = user;
```

It represents how your data looks

- **Routes/**

This defines API endpoints and connects them to controllers.

Eg:- **Without Middleware**

```
// routes/user.routes.js
const express = require("express");
const router = express.Router();
const userController = require("../controllers/user.controller");
router.get("/users", userController.getUsers);
module.exports = router;
```

Eg:- **With Middleware**

```
// routes/user.routes.js
const express = require("express");
const router = express.Router();
const userController = require("../controllers/user.controller");
const authMiddleware = require("../middleware/auth");
```

```
// Apply middleware to specific route
router.get("/users", authMiddleware, userController.getUsers);
// Without middleware
router.post("/users", userController.createUser)
module.exports = router;
```

It decides which function runs for which URL

What's Happening Now (Clear Flow)

When request comes: /api/users

- goes to **router**
- middleware runs first (authMiddleware)
- if passed → controller runs
- response sent

- **App.js**

This is where you configure your Express app.

Eg:-

```
const express = require("express");
const app = express();
app.use(express.json());
const userRoutes = require("./routes/user.routes");
app.use("/api", userRoutes);
module.exports = app;
```

It connects middleware and routes

- **Server.js**

This file starts the server.

Eg:-

```
const app = require("./app");
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Without this, your app won't run

- **Utils/**

Reusable helper functions used across the project.

Eg:-

```
// utils/response.js
exports.success = (res, data) => {
  res.json({ status: "success", data });
};
```

Avoids repeating code

- **Middleware/**

Functions that run before the controller (for validation, auth, etc.)

```
Eg:-    // middleware/auth.js
        module.exports = (req, res, next) => {
          console.log("Auth middleware called");

          // Example check
          const token = req.headers.authorization;
          if (!token) {
            return res.status(401).json({ message: "Unauthorized" });
          }
          next(); // move to next step (controller)
        };

```

Controls request flow before logic executes

- **Services/**

Contains reusable core logic like JWT creation, sending emails, etc.

```
Eg:-    // services/jwt.js
        const jwt = require("jsonwebtoken");
        exports.generateToken = (user) => {
          return jwt.sign(user, "secretkey");
        };

```

Keeps controllers clean

- **Config/**

Stores configuration like database connection and environment variables.

```
Eg:-    // config/db.js
        module.exports = {
          url: "mongodb://localhost:27017/mydb"
        };

```

Central place for setup values

- **Package.json**

In a Node.js project, the package.json file is the **main configuration file**. It stores information about your project and manages dependencies (libraries you install).

When you run:

- npm init -y
- package.json file is automatically created.

```
Eg:-    {
          "name": "my-app",
          "version": "1.0.0",

```

```

    "main": "server.js",
    "scripts": {
      "start": "node server.js"
    },
    "dependencies": {
      "express": "^4.18.2"
    }
  }
}

```

What it means (simple idea)

- **name** → project name
- **version** → version of your app
- **main** → entry file
- **scripts** → shortcuts to run commands
- **dependencies** → installed packages (like Express)

Instead of typing `node server.js` (Your file name),
you can run: **npm start**

8.4 CommonJS vs ES Modules (Important Concept)

By default, Node.js uses **CommonJS** module system in package.json file.

- CommonJS (default)
- ES Modules (Modern JavaScript)
 - To use this, add in package.json: `"type": "module"`
 - Now you can write:


```

import express from "express";
export default myFunction;

```

Key Difference

- **require()** → CommonJS
- **import/export** → ES Modules

You **cannot mix both** in the same file (this is where beginners break things).

9. Commonly Used Third-Party Dependencies in Node.js

In a Node.js project, third-party dependencies help speed up development by providing ready-made solutions for common tasks like server setup, authentication, validation, and more.

- **Express**
 - Used to build web servers and APIs
 - Handles routing, requests, and responses easily

- **Nodemon**
 - Automatically restarts the server when code changes
 - Saves time during development (no manual restart)
- **Dotenv**
 - Loads environment variables from a `.env` file
 - Used to store sensitive data (API keys, DB URLs)
- **Mongoose**
 - Helps interact with MongoDB easily
 - Provides schema-based data modeling
- **Jsonwebtoken (JWT)**
 - Used to create and verify authentication tokens
 - Commonly used for login systems and secure APIs
- **Bcrypt**
 - Used to hash passwords securely
 - Prevents storing plain-text passwords
- **Cors**
 - Enables cross-origin requests
 - Allows frontend and backend to communicate
- **Uuid**
 - Generates unique IDs
 - Useful for identifiers without database auto-ID
- **Multer**
 - Handles file uploads (images, documents)
 - Used in forms with file input
- **Axios**
 - Used to make HTTP requests to external APIs
 - Works like fetch but with more features

10. CRUD with Register & Login (Basic Project)

We'll build a simple API using **Node.js** + **Express.js** + **MongoDB**.

10.1 Install Required Dependencies:-

- **npm init -y** --- Create Project
- **npm install express mongoose dotenv bcrypt jsonwebtoken**
- **npm install nodemon --save-dev**

10.2 Project Setup:-

- **Server.js**

```
const app = require("./app");
const mongoDB = require("./config/db.js");
mongoDB(); --- Connect DB
app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

- **App.js**

```
const express = require("express");
const app = express();
app.use(express.json());
const userRoutes = require("./routes/user.routes");
app.use("/api", userRoutes);
module.exports = app;
```

- **Database Connection > config > db.js**

```
const mongoose = require("mongoose");
mongoose.connect("mongodb://127.0.0.1:27017/myapp")
  .then(() => console.log("DB Connected"))
  .catch(err => console.log(err));
Import this in server.js: require("./config/db");
```

- **User Model > models > user.model.js**

```
const mongoose = require("mongoose");
const userSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String
});
module.exports = mongoose.model("User", userSchema);
```

- **Controller (Register, Login, CRUD) > controllers > user.controller.js**

```
const User = require("../models/user.model");
const bcrypt = require("bcrypt");
const jwt = require("jsonwebtoken");
```

```
// Register
exports.register = async (req, res) => {
```



```

const { name, email, password } = req.body;

const hashed = await bcrypt.hash(password, 10); --- Password is hashed
--- User.Create represents User created and store in DB
const user = await User.create({
  name,
  email,
  password: hashed
});

res.json(user);
};

// Login
exports.login = async (req, res) => {
  const { email, password } = req.body;

  const user = await User.findOne({ email });
  if (!user) return res.status(400).send("User not found");

  const isMatch = await bcrypt.compare(password, user.password);
  if (!isMatch) return res.status(400).send("Invalid password");

  const token = jwt.sign({ id: user._id }, "secret");

  res.json({ token });
};

// CRUD
exports getUsers = async (req, res) => {
  const users = await User.find();
  res.json(users);
};

exports.updateUser = async (req, res) => {
  const user = await User.findByIdAndUpdate(req.params.id, req.body, { new: true });
  res.json(user);
};

exports.deleteUser = async (req, res) => {
  await User.findByIdAndDelete(req.params.id);
  res.send("User deleted");
};

```

- **Routes > routes > user.routes.js**

```
const express = require("express");
const router = express.Router();

const userController = require("../controllers/user.controller");

router.post("/register", userController.register);
router.post("/login", userController.login);

router.get("/users", userController.getUsers);
router.put("/users/:id", userController.updateUser);
router.delete("/users/:id", userController.deleteUser);

module.exports = router;
```

- **Run the Project**

`npm run dev`

- **API Endpoints**

- POST **/api/register** → Register user
- POST **/api/login** → Login user
- GET **/api/users** → Get all users
- PUT **/api/users/:id** → Update user
- DELETE **/api/users/:id** → Delete user

- **Final Flow Understanding**

- Register → Save user in DB
- Login → Verify + generate token
- CRUD → Read / Update / Delete data

- **What is devDependencies?**

DevDependencies are packages used **only during development**, not needed when running the app in production.

Eg:-

```
{
  "devDependencies": {
    "nodemon": "^3.0.0"
  }
}
```

11. Environment Variables (.env) in Node.js

In Node.js, environment variables are used to store **sensitive or configurable data** outside your code, such as database URLs, API keys, and secret tokens.

Instead of writing values directly in your code, you store them in a `.env` file and access them securely.

Why Use `.env`?

- Keeps sensitive data safe
- Avoids hardcoding values
- Easy to change config without modifying code

Step 1: Install `dotenv`;

```
npm install dotenv
```

Step 2: Create `.env` File

```
PORT=3000
DB_URL=mongodb://127.0.0.1:27017/myapp
JWT_SECRET=mysecretkey
```

Step 3: Load `.env` in Your Project

In your main file (`server.js` or `app.js`):

```
require("dotenv").config();
```

Step 4: Use Environment Variables

```
const port = process.env.PORT;
app.listen(port, () => {
  console.log(`Server running on port ${port}`);
});
```

Database example: `mongoose.connect(process.env.DB_URL);`

JWT example: `jwt.sign({ id: user._id }, process.env.JWT_SECRET);`

Best Practice

- Never upload `.env` file to GitHub
- Add `.env` to `.gitignore`

Simple Understanding

- `.env` = place to store secret/config values
- `dotenv` = loads those values into your app

